DBMS – ADVANCED SQL QUERY SYSTEM

SWIGGY INSTAMART E-COMMERCE DATABASE MANAGEMENT SYSTEM

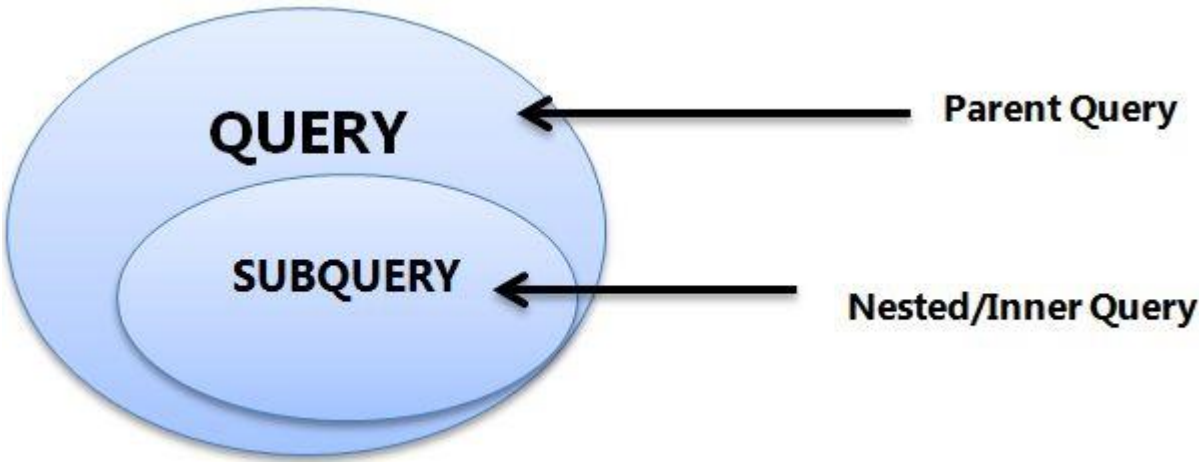
Objectives

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query.

The inner query is executed first, and its result is used by the outer query



Basic Structure

```
SELECT
    employeeid, firstname, lastname
FROM
    employees
WHERE
    employeeid IN (
        SELECT DISTINCT
            reportsto
        FROM
            employees);
```

outer query

subquery

Types of Subqueries

SUBQUERY

Types of subquery:

- Single row subquery
- Multiple row subquery
- Multiple column subquery
- Correlated subquery

Example

Find orders whose amount is greater than the average order amount.

```
SQL> SELECT
  2      PRODUCT_ID,
  3      PRODUCT_NAME,
  4      PRICE
  5  FROM Product
  6  WHERE PRICE > (
  7      SELECT AVG(PRICE)
  8      FROM Product
  9  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
---
     PRICE
----------
         3
Apple
       120

         5
Fruit Juice
        60

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
---
     PRICE
----------
```

Explanation:

- The inner query calculates the average price of all products.
- The outer query displays products whose price is greater than the average price.
- It shows the Product ID, Product Name and Price of those products.

2. Find Products Above Average Price

To display products whose price is greater than the average price of all products.

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price <
  4        (SELECT AVG(Price)
  5         FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
---
     PRICE
----------
         1
Milk
        30

         2
Bread
        40

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
---
     PRICE
----------

         4
Biscuits
        30
```

Explanation:

- The inner query calculates the average price of all products.
- The outer query displays products whose price is above the average price.

3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase amount.

Step 1: Calculate Customer-wise Purchase

```
SQL> SELECT Customer_ID,
  2         SUM(Total_Amount) AS Total_Purchase
  3  FROM Orders
  4  GROUP BY Customer_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101           1050
        102           1725
        103            750
        104           1340
        105           1970
```

Step 2: Find the Maximum Purchase

```
SQL> SELECT MAX(Total_Purchase) AS Maximum_Purchase
  2  FROM
  3  (
  4      SELECT SUM(Total_Amount) AS Total_Purchase
  5      FROM Orders
  6      GROUP BY Customer_ID
  7  );

MAXIMUM_PURCHASE
----------------
            1970
```

The inner query calculates the total purchase for each customer.

The outer query finds the maximum value among those totals

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name,
  3         C.Last_Name,
  4         O.Total_Purchase
  5  FROM Customer C
  6  JOIN
  7  (
  8      SELECT Customer_ID,
  9             SUM(Total_Amount) AS Total_Purchase
 10      FROM Orders
 11      GROUP BY Customer_ID
 12  ) O
 13  ON C.Customer_ID = O.Customer_ID
 14  WHERE O.Total_Purchase =
 15  (
 16      SELECT MAX(Total_Purchase)
 17      FROM
 18      (
 19          SELECT SUM(Total_Amount) AS Total_Purchase
 20          FROM Orders
 21          GROUP BY Customer_ID
 22      )
 23  );

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_PURCHASE
-------------------------------------------------- --------------
        105 Vikram
Singh                                                        1970
```

4. Complex Business Queries

To perform complex business-related queries using subqueries, joins and aggregate functions to analyse business data.

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name,
  3         C.Last_Name,
  4         O.Order_ID,
  5         O.Total_Amount
  6  FROM Customer C
  7  JOIN Orders O
  8  ON C.Customer_ID = O.Customer_ID
  9  WHERE O.Total_Amount >
 10        (SELECT AVG(Total_Amount)
 11         FROM Orders);

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          ORDER_ID TOTAL_AMOUNT
-------------------------------------------------- ---------- ------------
        102 Priya
Kumar                                                1002          850

        102 Priya
Kumar                                                1007          875

        104 Sneha
Iyer                                                 1009          690


CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          ORDER_ID TOTAL_AMOUNT
-------------------------------------------------- ---------- ------------
        105 Vikram
Singh                                                1005          920
```

Purpose:

This query identifies customers who placed orders whose value is greater than the overall average order value. It helps the business identify high-value orders and understand customer purchasing behaviour.

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated from Order_Item.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         SUM(OI.Quantity * OI.Unit_Price) AS Total_Sales
  4  FROM Product P
  5  JOIN Order_Item OI
  6  ON P.Product_ID = OI.Product_ID
  7  GROUP BY P.Product_ID, P.Product_Name
  8  HAVING SUM(OI.Quantity * OI.Unit_Price) >
  9         (SELECT AVG(Product_Sales)
 10          FROM
 11          (
 12              SELECT SUM(Quantity * Unit_Price) AS Product_Sales
 13              FROM Order_Item
 14              GROUP BY Product_ID
 15          ));

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
---
TOTAL_SALES
-----------
         2
Bread
        900

         4
Biscuits
       1000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
---
TOTAL_SALES
-----------
```

Purpose:

This query identifies products whose total sales are above average. It helps the business find the best-selling products and analyse sales performance.

6. Customer Purchase Report

A detailed customer purchase report can combine multiple tables.

```
SQL> SELECT C.Customer_ID,
  2         C.First_Name,
  3         C.Last_Name,
  4         COUNT(O.Order_ID) AS Number_Of_Orders,
  5         SUM(O.Total_Amount) AS Total_Purchase,
  6         AVG(O.Total_Amount) AS Average_Order_Value
  7  FROM Customer C
  8  JOIN Orders O
  9  ON C.Customer_ID = O.Customer_ID
 10  GROUP BY C.Customer_ID, C.First_Name, C.Last_Name
 11  ORDER BY Total_Purchase DESC;

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          NUMBER_OF_ORDERS
-------------------------------------------------- ---------------
TOTAL_PURCHASE AVERAGE_ORDER_VALUE
-------------- -------------------
         105 Vikram
Singh                                                             2
          1970                985

         102 Priya
Kumar                                                             2
          1725               862.5

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          NUMBER_OF_ORDERS
-------------------------------------------------- ---------------
TOTAL_PURCHASE AVERAGE_ORDER_VALUE
-------------- -------------------

         104 Sneha
Iyer                                                              2
          1340                670
```

Report Contains:

- Customer ID
- Customer Name
- Number of Orders
- Total Purchase Amount
- Average Order Value

7. Advanced Product Sales Report

This report combines product and order item.

```
SQL> SELECT P.Product_ID,
  2         P.Product_Name,
  3         COUNT(OI.Order_ID) AS Number_Of_Orders,
  4         SUM(OI.Quantity) AS Total_Quantity_Sold,
  5         SUM(OI.Quantity * OI.Unit_Price) AS Total_Sales,
  6         AVG(OI.Unit_Price) AS Average_Selling_Price
  7  FROM Product P
  8  JOIN Order_Item OI
  9  ON P.Product_ID = OI.Product_ID
 10  GROUP BY P.Product_ID, P.Product_Name
 11  ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
---
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
---------------- ------------------- ----------- ---------------------
         4
Biscuits
               2                   5        1000                   200


         5
Fruit Juice
               2                   3         960                   320

PRODUCT_ID
----------
PRODUCT_NAME
```

Purpose

This report helps the business identify:

• Number of orders for each product

• Total quantity sold

• Total sales

• Average selling price

• Products generating higher sales

8. Category-wise Advanced Sales Report

Category-wise sales can be generated by combining Category, Product, and Order_Item.

```
SQL> SELECT C.Category_ID,
  2         C.Category_Name,
  3         SUM(OI.Quantity) AS Total_Quantity_Sold,
  4         SUM(OI.Quantity * OI.Unit_Price) AS Total_Sales
  5  FROM Category C
  6  JOIN Product P
  7  ON C.Category_ID = P.Category_ID
  8  JOIN Order_Item OI
  9  ON P.Product_ID = OI.Product_ID
 10  GROUP BY C.Category_ID, C.Category_Name
 11  ORDER BY Total_Sales DESC;

CATEGORY_ID CATEGORY_NAME
----------- --------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
        104 Snacks
                 11        1900

        103 Beverages
                  3         960

        105 Dairy
                  5         780


CATEGORY_ID CATEGORY_NAME
----------- --------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
        101 Fruits
                  5         400
```